

LANL Threat Prediction Pipeline — Project 2 Write-Up

Team: Mark Crisci Live URL: <https://lanlthreat.streamlit.app> Repo: <https://github.com/mcrisci24/lanl-threat-pipeline>

1. Prediction question

Given a single computer's behavior in the current event-time window (authentication, network flows, DNS, process events), will the same computer show red-team activity in the next window? The label is built with a Spark window function that shifts the red-team flag forward by one window, and every current-window red-team feature is explicitly removed from the feature set so the model cannot peek at the answer.

2. Data source

The Los Alamos National Laboratory enterprise telemetry dataset — five gzipped event streams (auth, flows, dns, proc, redteam) totalling roughly 11 GB compressed — covering relative event time across an internal network. Raw files land in S3 under `s3:///lanl/bronze/`. Refresh is one-time for this dataset, but the medallion partitioning supports incremental writes if more days were added.

3. Pipeline architecture

A medallion architecture on AWS, with three of the four pipeline layers running on distributed/cloud infrastructure (spec requires ≥ 2):

- Storage / ingestion: raw .txt.gz in S3 bronze/.
- Transformation (distributed): PySpark on AWS EMR — `bronze_to_silver_emr.py` applies explicit schemas and writes per-source silver parquet partitioned by event-day. `silver_to_gold_emr.py` aggregates each source by (computer, time_window), joins them, builds derived ratios, and creates the future-window label with a Spark window function.
- Clean storage: Parquet on S3 (silver/, gold/), partitioned for cheap reads.
- Training + tracking: scikit-learn pipeline trained from gold; every run logged to MLflow (metrics, parameters, leakage metadata, artifact). The winning run is registered under `lanl_threat_predictor` and aliased Production in the MLflow Model Registry.
- Serving: FastAPI on EC2 (/health, /model_info, /features, /predict, /batch_predict).
- UI: Streamlit with preset scenarios, a verdict gauge, a probability chart, and a metrics tab.

4. Model approach

Two scikit-learn pipelines (median impute → standard scale → classifier): a logistic-regression baseline and a balanced random forest with 300 trees. Inputs are ~40 engineered behavioral features (auth, flows, DNS, proc, plus derived ratios like `auth_failure_ratio` and `flows_bytes_per_event`). The split is time-aware — earliest 60 % windows train, next 20 % validate, last 20 % test — so the model is evaluated on windows that strictly follow its training data. Reported metrics: precision, recall, F1, ROC AUC, and PR AUC. PR AUC is the headline because red-team windows are rare; accuracy is misleading on this imbalance. The best validation-F1 model (typically the random forest) is what the API serves.

5. Learnings

The biggest realization was how easy it is to ship a model that looks great on paper and is actually cheating. Our first attempts had `redteam_flag` leaking through the same-window aggregates and validation F1 was near 1.0. Rebuilding the target as the next window's red-team status and asserting at training time that no current-window red-team column survives feature selection dropped the metrics to honest numbers and made the predictions actually mean something. The other lesson was infrastructure pragmatism: we started on Databricks, hit free-tier storage walls on an 11 GB dataset, and pivoted to S3 + EMR + EC2 without losing the medallion architecture or the MLflow story. If we had more time we'd add a streaming layer (Kinesis → Spark Structured Streaming) and an EventBridge-driven retraining schedule.